Undecidable Proof Strategies

So far, we have learned about undecidability and the *reduction* technique which allows us to show that a problem is solvable given another problem. In this unit, we use reductions to establish that one problem is undecidable by showing that it reduces to another problem known to be undecidable. We have some intuition of what a reduction is—solving the second problem by using a helper function that solves the first problem. However, it is not straightforward to map that intuition into a strategy for building reducibility proofs. In this reading, we outline a particular strategy for developing these proofs so that they are less magical and more mechanical.

An Example Reduction

Consider the following language:

$$\mathsf{ALL_{TM}} = \{\langle M \rangle \mid M \text{ is a TM and } L(M) = \Sigma^*\}$$

To prove that this language is undecidable, we will reduce a known, undecidable language to it. A common choice that we'll use for this problem is $\mathsf{A_{TM}}$:

$$\mathsf{A_{TM}} = \{\langle M, w \rangle \mid M \text{ is a TM and } M \text{ accepts } w\}$$

Thus our goal is to show that $\mathsf{A_{TM}} \leq \mathsf{ALL_{TM}}$, that is, $\mathsf{A_{TM}}$ is reducible to $\mathsf{ALL_{TM}}$.

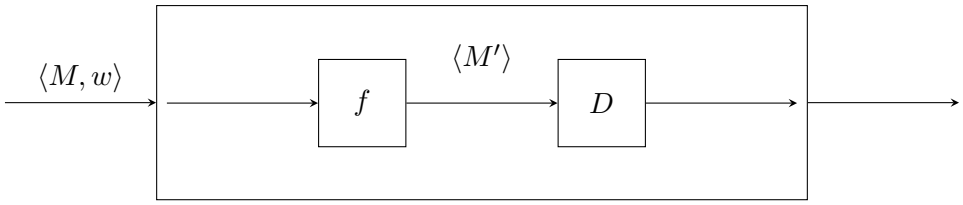
Recall that (mapping) reducibility says that to establish this fact, we need to create a function f that maps a string w in $\mathsf{A_{TM}}$ to a string in $\mathsf{ALL_{TM}}$ such that:

1. f itself is computable (*i.e.*, halts on every input) and
2. $w \in \mathsf{A_{TM}} \iff f(w) \in \mathsf{ALL_{TM}}$

$\mathsf{A_{TM}}$ accepts TM descriptions and inputs $\langle M, x \rangle$ and A accepts TM descriptions $\langle M \rangle$. Therefore, our function can be written as $f(\langle M, w \rangle) = \langle M' \rangle$ where:

$$M \text{ accepts } w \iff L(M') = \Sigma^*.$$

Note that all we need to do is create this function f to complete the proof. This is because we are using f in a standard way to build a decider for $\mathsf{A_{TM}}$ given a decider D for $\mathsf{ALL_{TM}}$:



Our decider for $\mathsf{A_{TM}}$, the outer-most box, proceeds by transforming its input using f and then runs that output through D. The decider then outputs whatever D outputs. This decides $\mathsf{A_{TM}}$ but is a contradiction because we know $\mathsf{A_{TM}}$ is not decidable. Therefore, we can conclude that D must not exist, *i.e.*, A is not decidable.

Now, our proof consists of constructing f with the two properties listed above. To do so, I find it best to operate in three steps:

This work is licensed under a "CC BY-NC-SA 4.0" license.

1. As a starting point, build a TM P that it obeys the right-hand side of the biconditional and a TM Q that does not obey this property. We will base M' off of the behavior of these *target* TMs.
2. Next, construct M', the output of f, that conditionally acts like P if the left-hand side of the biconditional is true and Q if the left-hand side of the biconditional is false.
3. Verify that this M' fully obeys the two conditions of mapping reducibility.

For simple reducibility proofs, this construction strategy allows us to quickly build an appropriate mapping function f. More complicated proofs may require additional manipulation, but this strategy at least gives us a starting point in our investigation.

For our example, we must first build a simple machine P such that $L(P) = \Sigma^*$. This is straightforward to do:

$P =$"On input x:
1. Accept."

P accepts any string x given to it, therefore, its language is Σ^*.

Let's also build a simple machine Q such that $L(Q) \neq \Sigma^*$. We have more options here, *i.e.*, Q could accept some finite number of strings. However, it is simpler to have Q simply accept nothing:

$Q =$"On input x:
1. Reject."

Q rejects any string x given to it, therefore, its language is $\emptyset$.

Next, let's build M' that is the output of our mapping function $f(\langle M, w\rangle)$. Based on our biconditional, whenever M accepts w, we should emulate P's behavior; otherwise, we emulate Q's behavior. Define our mapping function as $f(\langle M, w\rangle) = \langle M'\rangle$ where:

$M' =$"Ignore the input.
1. Run M on w.
2. If M accepts, **accept**.
3. If M rejects, **reject**."

Finally, let's check that M' has the appropriate behavior:

- If M accepts w, then the conditional of M' is always true, therefore the machine always accepts. Thus $L(M') = \Sigma^*$.
- If M rejects w then the conditional of M' is always false, therefore the machine always rejects. Thus $L(M') = \emptyset \neq \Sigma^*$.
- If M loops on w then M' never proceeds past line 1 and thus never accepts any string. Thus $L(M') = \emptyset \neq \Sigma^*$.

We can illustrate this three-stage procedure by filling in a table. The contents of the table relate both of the languages in question, as well as how the inputs for each should behave (one assumed, and one constructed).

Behavior of M, w	$\langle M, w\rangle \stackrel{?}{\in} \mathsf{A_{TM}}$	$\langle M'\rangle \stackrel{?}{\in} \mathsf{ALL_{TM}}$	Behavior of M'	M' should...

This work is licensed under a "CC BY-NC-SA 4.0" license.

We first fill in the two columns corresponding to the condition that $\langle M, w\rangle \in \mathsf{A_{TM}} \iff f(\langle M, w\rangle \in \mathsf{ALL_{TM}})$, recalling that M' is the Turing machine we will be constructing as the input to the decider D for $\mathsf{ALL_{TM}}$.

Behavior of M, w	$\langle M, w\rangle \stackrel{?}{\in} \mathsf{A_{TM}}$	$\langle M'\rangle \stackrel{?}{\in} \mathsf{ALL_{TM}}$	Behavior of M'	M' should...
	$\langle M, w\rangle \in \mathsf{A_{TM}}$	$\langle M'\rangle \in L$		
	$\langle M, w\rangle \notin \mathsf{A_{TM}}$	$\langle M'\rangle \notin L$		

Based on these memberships, we can say something about what the intended behaviors of M, w, and, in this case, M' should be. Note that the specific details here will depend upon the languages in any particular reduction, *e.g.*, the types of inputs and which are accepted may vary.

Behavior of M, w	$\langle M, w\rangle \stackrel{?}{\in} \mathsf{A_{TM}}$	$\langle M'\rangle \stackrel{?}{\in} \mathsf{ALL_{TM}}$	Behavior of M'	M' should...
M accepts w	$\langle M, w\rangle \in \mathsf{A_{TM}}$	$\langle M'\rangle \in L$	$L(M') = \Sigma^*$	
M doesn't accept w	$\langle M, w\rangle \notin \mathsf{A_{TM}}$	$\langle M'\rangle \notin L$	$L(M') = \emptyset$	

Finally, based on the entries so far, we can determine how we should construct M'. This is where we want to think about machines P and Q and how to combine them.

Behavior of M, w	$\langle M, w\rangle \stackrel{?}{\in} \mathsf{A_{TM}}$	$\langle M'\rangle \stackrel{?}{\in} \mathsf{ALL_{TM}}$	Behavior of M'	M' should...
M accepts w	$\langle M, w\rangle \in \mathsf{A_{TM}}$	$\langle M'\rangle \in L$	$L(M') = \Sigma^*$	act like P
M doesn't accept w	$\langle M, w\rangle \notin \mathsf{A_{TM}}$	$\langle M'\rangle \notin L$	$L(M') = \emptyset$	act like Q

It can sometimes be tricky to ensure that we get the desired behavior in both the "M rejects w" and the "M loops on w" cases. One tip that can help make this more clear is that you can actually split these into separate rows in the table!

Behavior of M, w	$\langle M, w\rangle \stackrel{?}{\in} \mathsf{A_{TM}}$	$\langle M'\rangle \stackrel{?}{\in} \mathsf{ALL_{TM}}$	Behavior of M'	M' should...
M accepts w	$\langle M, w\rangle \in \mathsf{A_{TM}}$	$\langle M'\rangle \in L$	$L(M') = \Sigma^*$	act like P
M rejects w	$\langle M, w\rangle \notin \mathsf{A_{TM}}$	$\langle M'\rangle \notin L$	$L(M') = \emptyset$	act like Q
M loops on w	$\langle M, w\rangle \notin \mathsf{A_{TM}}$	$\langle M'\rangle \notin L$	$L(M') = \emptyset$	always loop

Here, we have that M' should loop on any string if M loops on w (it will be very difficult to do anything else in this case). One final note: More complicated reductions may have additional conditions you would like to implement in the "output" of the reduction f. This could correspond to either making the column headers more precise, or even adding additional columns!

Snags in Reductions

While there are many potential errors that can arise when constructing mapping reductions between languages concerning Turing machines, there are a few common types we highlight here.

You should ensure that the types of the input/output of the reduction are correct. Since we are often working with Turing machines, this typically means that the ouput is of the appropriate form (often $\langle M\rangle, \langle M, w\rangle, \langle M_1, M_2\rangle$, etc.). This issue can arise in subtle ways: a conditional of the form "If M accepts w do X, and otherwise do Y" hides an implicit check in the second case. To reach this case, we would need to be able to determine that M does not accept w. But we know this is unrecognizable! So there is no Turing machine that can actually implement such a check, even as a

This work is licensed under a "CC BY-NC-SA 4.0" license.

subroutine. To avoid this problem, I suggest actively avoiding situations where the word "otherwise" would naturally arise in your mapping reductions. The above conditional would instead be "If M accepts w do X, and if M does not accept w do Y," from which it is more clear that the condition you are trying to check for is unrecognizable and thus not implementable.

Similarly, we require that the mapping reduction f is *computable*. This means that any subroutine used in a reduction has to be a computation that can be performed on a Turing machine. That is, the procedure we use to *generate* the output of the reduction has to halt. Note carefully: This is not the same as requiring that the *output* of the reduction is a Turing machine that always halts. Again, telltale signs that a proposed reduction is not actually computable are often conditionals that depend upon unrecognizable behavior. For example, statements of the form "Run M on w and then do X" will have either undefined or unexpected behavior, because this implicitly assumes that M halts on w in order to reach the "then" portion. Such a dependence could again be made more clear by splitting into the cases: "If M accepts w do X, if M rejects w do Y, and if M loops on w do Z" and handled by considering carefully the fact that the "do Z" branch will never be reached.

One constraint to keep in mind for correctness is that, in reductions from languages like $\mathsf{A_{TM}}$, we must ensure that our newly constructed Turing machine has similar behavior in the cases where "the original TM M rejects w" and "the original TM M diverges on w." They do not need to behave *identically*; however, they should both lead to the $\langle M' \rangle \notin L$ case. In the example reduction from earlier, we could have just as well made M' accept only ε in the case where M rejects w. Then these two cases would have slightly different behavior, although neither would result in M' recognizing Σ^*. This can sometimes be difficult to enforce for reasons related to the first sticking point: We can't actually make a Turing machine condition on "M diverges on w!" So it can often be useful to reason about "how does my construction treat the case where M diverges on w" and *choose* its intended behavior in the accept/reject cases to be distinct and similar, respectively. An explicit example of the types of behavior here: If you know that you want some string x to be accepted in the case that M diverges on w, this string must actually be accepted in all cases! So it would be good to structure your reduction so that the language of Q is a subset of the language of P, and just have M' immediately accept every string in Q (provided that Q is decidable).

Finally, there may be situations where you want to construct a Turing machine to conditionally have a particular language. As an example, you may want $L(M') = \{x\}$, the singleton language containing only the string x. It is incorrect to write "On input w, have the Turing machine accept x." This is because the value that M' accepts *must be* its input; if the input is not x, the machine's execution on this input has no bearing on its acceptance of x. To produce this behavior, it is better to set it up with a conditional of the form "On input w, if $w = x$, accept."

Acknowledgments

This reading was written by Dr. Peter-Michael Osera and Dr. Shawn Ong.

This work is licensed under a "CC BY-NC-SA 4.0" license.